VERS UN CODE IMMUABLE

Par Pierre, Martial et Roman

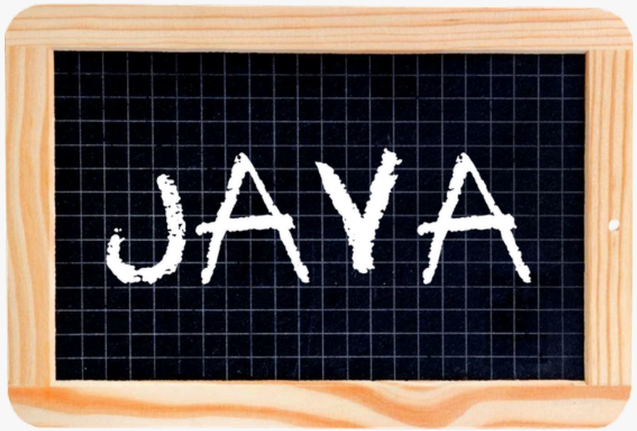
LE SOMMAIRE

1. **INTRODUCTION À L'IMMUABILITÉ**
   a. Qu'est-ce qu'un état immuable ?
   b. Pourquoi s'y intéresser ?
   c. Avantages et limites
2. **METTRE EN ŒUVRE L'IMMUABILITÉ EN JAVA CLASSIQUE**
   a. La « recette » pas à pas
   b. Le cas critique des collections
3. **LES OUTILS DE JAVA POUR SIMPLIFIER L'IMMUABILITÉ**
   a. Les Records
   b. Objects.requireNonNull()
   c. List.of() et List.copyOf()
   d. Immuabilité superficielle et profonde
   e. Les collections immuables persistantes avec VAVR
4. **L'IMMUABILITÉ EN CONCEPTION MÉTIER**
   a. Deux patterns fondamentaux : Value Object vs Entity
   b. Évolution d'un Value Object : de la classe verbeuse au record
5. **DÉMONSTRATION PRATIQUE & ÉVALUATION**
   a. Live coding
   b. QCM interactif
6. **IAGRAPHIE ET SITOGRAPHIE**

Qu’est-ce qu’un état immuable ?

OBJET MUTABLE :

son état interne peut changer après instanciation (méthodes modificatrices / setters).



OBJET IMMUABLE :

son état est figé dès sa création en mémoire. Toute modification produit une nouvelle instance.



« Code immuable : on ne modifie jamais un objet existant, on en crée un nouveau. »

« Pourquoi s’y intéresser ? »

- **« Qui a modifié mon objet ? »**
- **« Pourquoi mon résultat change selon l’ordre d’exécution ? »**
- **« Pourquoi mon objet a disparu de ma HashMap ? »**

Trois bugs, une même cause : un état qui change sans qu’on le voie.

UN EXEMPLE POUR COMPRENDRE :

OBJET MUTABLE :

```java
class Panier {
    private int total;

    public Panier(int total) { this.total = total; }
    public int getTotal() { return total; }
    public void setTotal(int total) { this.total = total; }
}
```

```java
Panier commandeValidee = new Panier(50);

// Un service applique une promo sur la référence partagée
Panier panierPromo = commandeValidee;
panierPromo.setTotal(30);

System.out.println(commandeValidee.getTotal()); // 30 au lieu de 50 !
```

OBJET IMMUABLE :

```java
// Classe immuable
final class PanierImmuable {
    private final int total;

    public PanierImmuable(int total) { this.total = total; }
    public int getTotal() { return total; }

    // Renvoie une NOUVELLE instance
    public PanierImmuable avecReduction(int remise) {
        return new PanierImmuable(this.total - remise);
    }
}
```

```java
// Utilisation
PanierImmuable commandeValidee = new PanierImmuable(50);
PanierImmuable panierPromo = commandeValidee.avecReduction(20);

System.out.println(commandeValidee.getTotal()); // Reste à 50 (intact !)
System.out.println(panierPromo.getTotal());       // Vaut 30 (nouvelle instance)
```

Même besoin, deux comportements

Les avantages

- **Prévisibilité** : aucune méthode ne peut modifier l'objet en douce.
- **Thread-safety native** : partageable entre threads sans synchronized (si l'objet est entièrement immuable).
- **Fiabilité dans les collections** : le hashCode ne change plus, donc l'objet reste retrouvable dans un HashSet ou une HashMap.
- **Testabilité** : état stable, tests déterministes.

Les limites et compromis

- **Coût d'instanciation :** chaque « modification » crée un objet. C'est souvent négligeable, sauf dans des boucles serrées (traitement d'images, calculs matriciels).
- **Verbosité en Java classique :** constructeur complet, copies défensives, equals, hashCode, toString à écrire à la main.
- **Pas adapté à tout :** un objet dont l'état évolue naturellement dans le métier (un compte, un panier en cours) se prête mal à l'immuabilité. On y revient avec Entity.

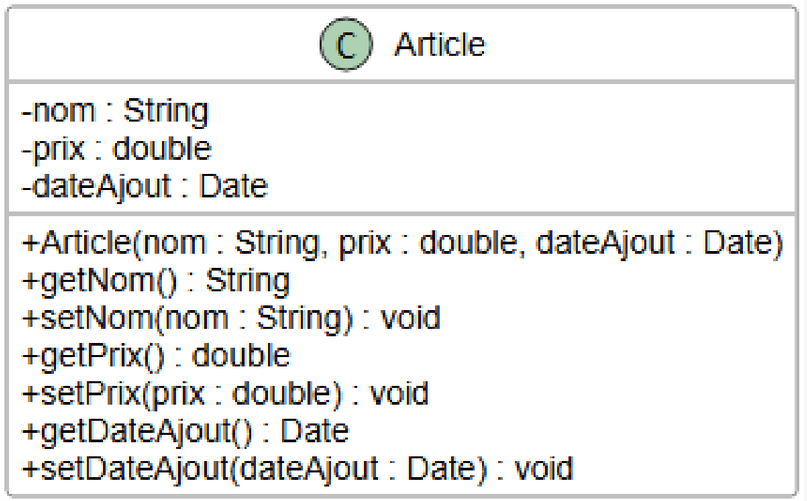
La recette : point de départ

```java
public class Article {
    private String nom;
    private double prix;
    private Date dateAjout;

    public Article(String nom, double prix, Date dateAjout) {
        this.nom = nom;
        this.prix = prix;
        this.dateAjout = dateAjout;
    }
```

```java
    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public double getPrix() { return prix; }
    public void setPrix(double prix) { this.prix = prix; }
    public Date getDateAjout() { return dateAjout; }
    public void setDateAjout(Date dateAjout) { this.dateAjout = dateAjout; }
}
```

- **Règle 1 :** classe final
- **Règle 2 :** attributs private final
- **Règle 3 :** aucun setter
- **Règle 4 :** copie défensive à l'entrée
- **Règle 5 :** copie défensive à la sortie
- **Règle 6 :** equals, hashCode, toString



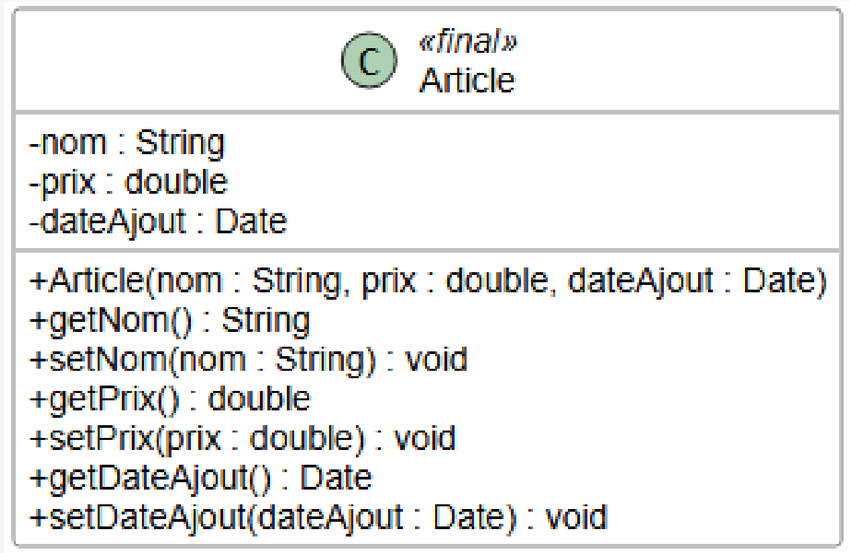
Règle 1 : la classe est final

- Une classe final ne peut pas être héritée.
- Aucune sous-classe ne peut ajouter des setters ou redéfinir les accesseurs.

```java
public final class Article {          // Règle 1
    private String nom;
    private double prix;
    private Date dateAjout;

    public Article(String nom, double prix, Date dateAjout) {
        this.nom = nom;
        this.prix = prix;
        this.dateAjout = dateAjout;
    }

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public double getPrix() { return prix; }
    public void setPrix(double prix) { this.prix = prix; }
    public Date getDateAjout() { return dateAjout; }
    public void setDateAjout(Date dateAjout) { this.dateAjout = dateAjout; }
}
```



Mais est-ce que Article est immuable pour autant ?

Règles 2 et 3 : attributs private final, aucun setter

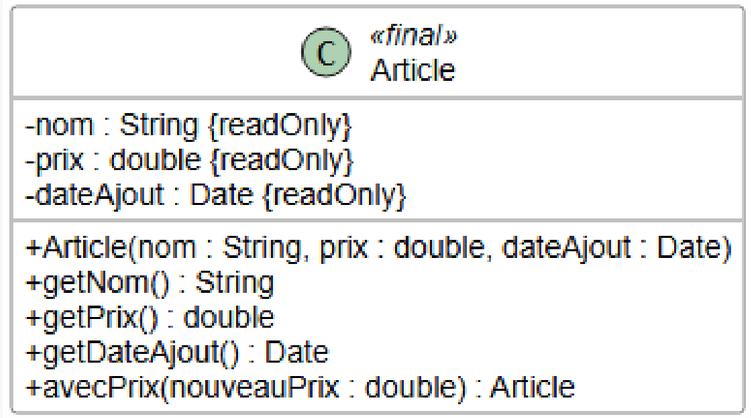
- Règle 2 : les attributs sont private final, donc affectés une seule fois, dans le constructeur.
- Règle 3 : aucun setter. Pour « modifier », on renvoie une nouvelle instance.

```java
public final class Article {
    private final String nom;         // Règle 2
    private final double prix;
    private final Date dateAjout;

    public Article(String nom, double prix, Date dateAjout) {
        this.nom = nom;
        this.prix = prix;
        this.dateAjout = dateAjout;
    }

    // Règle 3 : accesseurs uniquement
    public String getNom() { return nom; }
    public double getPrix() { return prix; }
    public Date getDateAjout() { return dateAjout; }

    // « Modifier » = créer une nouvelle instance
    public Article avecPrix(double nouveauPrix) {
        return new Article(nom, nouveauPrix, dateAjout);
    }
}
```



Plus aucun setter… mais regardons dateAjout.

Un attribut final peut quand même être modifié

- final empêche de réaffecter la référence, pas de modifier l'objet pointé.
- Date est un type mutable : l'extérieur garde un accès à notre état.

```
Date d = new Date();
Article a = new Article("Livre", 12.5, d);

d.setTime(0);                    // modifie l'état interne de l'article
a.getDateAjout().setTime(0);     // idem, via l'accesseur
```

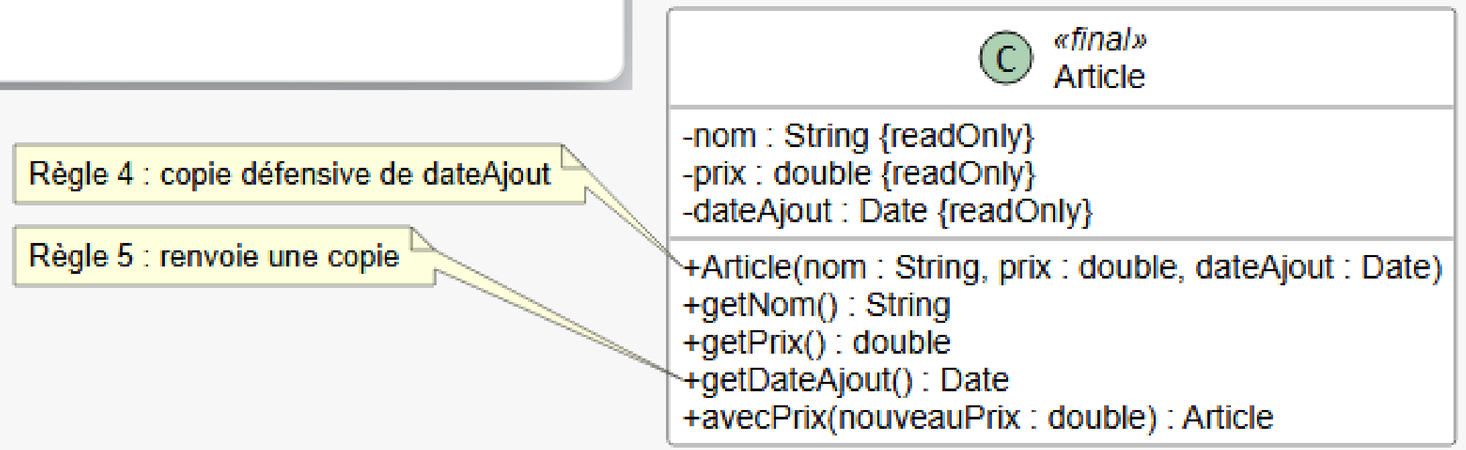
Deux fuites : une à l'entrée, une à la sortie.

Règles 4 et 5 : copies défensives

- **Règle 4** : copier à l'entrée, dans le constructeur.
- **Règle 5** : copier à la sortie, dans l'accesseur.
- Inutile pour String et double, déjà immuables.

```java
public Article(String nom, double prix, Date dateAjout) {
    this.nom = nom;
    this.prix = prix;
    this.dateAjout = new Date(dateAjout.getTime());   // Règle 4 : copie à l'entrée
}

public Date getDateAjout() {
    return new Date(dateAjout.getTime());             // Règle 5 : copie à la sortie
}
```



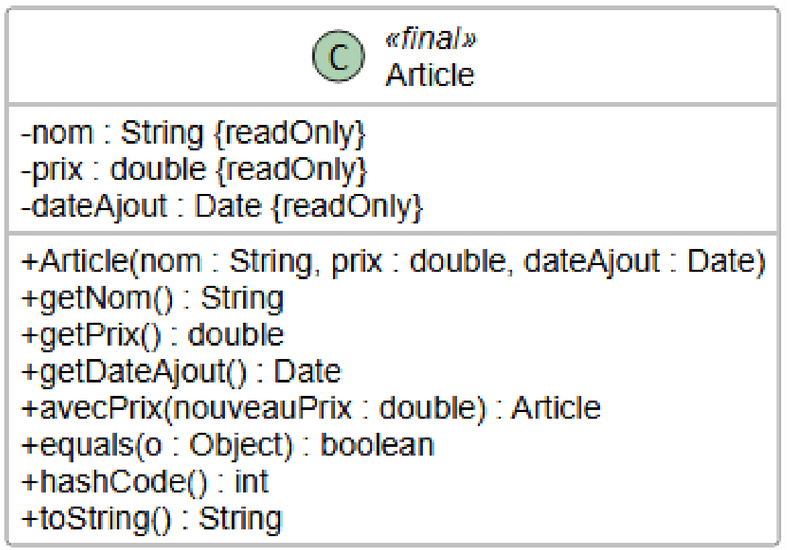
Règle 6 : equals, hashCode et toString

- Sans equals, deux articles identiques ne sont pas égaux (comparaison par référence).
- hashCode doit rester cohérent avec equals, sinon l'objet est mal retrouvé dans un HashSet ou une HashMap.
- C'est la partie la plus verbeuse de la recette. Les records, plus tard, la génèrent pour nous.

```java
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Article other = (Article) o;
    return Double.compare(prix, other.prix) == 0
            && nom.equals(other.nom)
            && dateAjout.equals(other.dateAjout);
}

@Override
public int hashCode() {
    return Objects.hash(nom, prix, dateAjout);
}

@Override
public String toString() {
    return "Article[nom=" + nom + ", prix=" + prix + ", dateAjout=" + dateAjout + "]";
}
```



Beaucoup de code pour une simple classe de données… Java moderne a une solution.

Le piège des collections (suite) : la fuite de référence

- Une classe final avec un attribut private final List n'est pas immuable.
- Le constructeur et l'accesseur laissent l'extérieur garder une référence vers la liste interne.

```java
public final class Commande {
    private final List<String> articles;

    public Commande(List<String> articles) { this.articles = articles; }
    public List<String> getArticles() { return articles; }
}
```

```java
List<String> liste = new ArrayList<>(List.of("Livre"));
Commande c = new Commande(liste);

liste.add("Stylo");          // fuite à l'entrée : la commande a changé
c.getArticles().clear();     // fuite à la sortie : la commande est vidée
```

Une collection standard n'est pas immuable parce que son API expose des méthodes de mutation (add, remove, clear…), pas parce que son contenu varie.

Modifiable, non modifiable ou immuable ?

Type	Modifiable ?	Remarque
ArrayList, HashMap, HashSet	Oui	Collections classiques
Collections.unmodifiableList(...)	Non, mais c'est une vue	Suit les changements de la liste source
List.of, Set.of, Map.of, List.copyOf	Non	Immuables en structure seulement
vavr, Guava ImmutableList	Non	append renvoie une nouvelle collection

```java
List<String> source = new ArrayList<>(List.of("Africa", "Asia"));
List<String> vue = Collections.unmodifiableList(source);

vue.add("Europe");          // UnsupportedOperationException
source.add("Europe");       // autorisé
System.out.println(vue);    // [Africa, Asia, Europe] : la vue a changé !
```

Corriger Commande

```java
public final class Commande {
    private final List<String> articles;

    public Commande(List<String> articles) {
        this.articles = List.copyOf(articles);   // copie à l'entrée, liste non modifiable
    }

    public List<String> getArticles() {
        return articles;   // déjà non modifiable : pas de copie à la sortie
    }
}
```

final class != classe immuable

```java
// Professor.java
public final class Professor {
    private String name;

    public Professor(String name) {
        this.name = name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

// Main.java
Professor professor = new Professor("Ms. Blasquez");
professor.setName("Mr. Paris"); // Allowed

```

Le record

Déclaration :

```java
package latice.model.board;

public record BoardMoveHint(int x, int y, int score) {}

```

Instanciation :

```java
List<BoardMoveHint> moveHints = new ArrayList<>();
moveHints.add(new BoardMoveHint(x, y, score));

```

Utilisation:

```java
public void highlightCellsWithScores(List<BoardMoveHint> moveHints) {
    // Some confidential code

    for (BoardMoveHint moveHint : moveHints) {
        highlightCell(
            moveHint.x(),
            moveHint.y(),
            moveHint.score()
        );
    }
}

```

Utilité du record

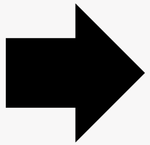
```java
public final class BoardMoveHint {
    private final int x;
    private final int y;
    private final int score;

    public BoardMoveHint(int x, int y, int score) {
        this.x = x;
        this.y = y;
        this.score = score;
    }

    public int x() {
        return x;
    }

    public int y() {
        return y;
    }

    public int score() {
        return score;
    }
}
```



```java
package latice.model.board;

public record BoardMoveHint(int x, int y, int score) {}
```

Constructeur compacte

```java
package latice.model.board;

public record BoardMoveHint(int x, int y, int score) {
    public BoardMoveHint {
        if (x < 0 || y < 0 || score < 0) {
            throw new IllegalArgumentException(
                "Values must be non-negative"
            );
        }
    }
}

```

Garantit la validité de l'état de l'objet dès sa création

Objects.requireNonNull()

```java
// Professor.java
import java.util.Objects;

public record Professor(int id, String name) {
    public Professor {
        Objects.requireNonNull(name, "Name cannot be null");
    }
}


// Main.java
Professor professor = new Professor(101, "Ms. Blasquez"); // Valid

Professor emptyName = new Professor(101, ""); // Valid

// Throws NullPointerException :
Professor invalidProfessor = new Professor(102, null);

```

Une Liste non modifiable

```java
final List<String> continents =
new ArrayList<>(List.of(
    "Africa",
    "America",
    "Asia",
    "Europe",
    "Oceania",
    "Antarctica"
));

continents.set(0, "Plastic Continent"); // Allowed
continents.add("Plastic Continent"); // Allowed

```

VS

```java
List<String> continents = List.of(
    "Africa",
    "America",
    "Asia",
    "Europe",
    "Oceania",
    "Antarctica"
);

// Throws UnsupportedOperationException
continents.add("Plastic Continent");

```

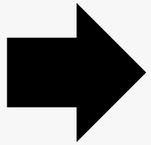
Les limites de List.copyOf()

```java
public final class Continent {
    private String name;

    public Continent(String name) {
        this.name = name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```



```java
List<Continent> continents = new ArrayList<>(List.of(
    new Continent("Africa"),
    new Continent("America"),
    new Continent("Asia"),
    new Continent("Europe"),
    new Continent("Oceania"),
    new Continent("Antarctica")
));

List<Continent> immutableContinents = List.copyOf(continents);

immutableContinents.add(new Continent("Plastic Continent"));
// Throws UnsupportedOperationException

immutableContinents.get(0).setName("Plastic Continent"); // Allowed
```

Immuabilité superficielle (shallow immutability)

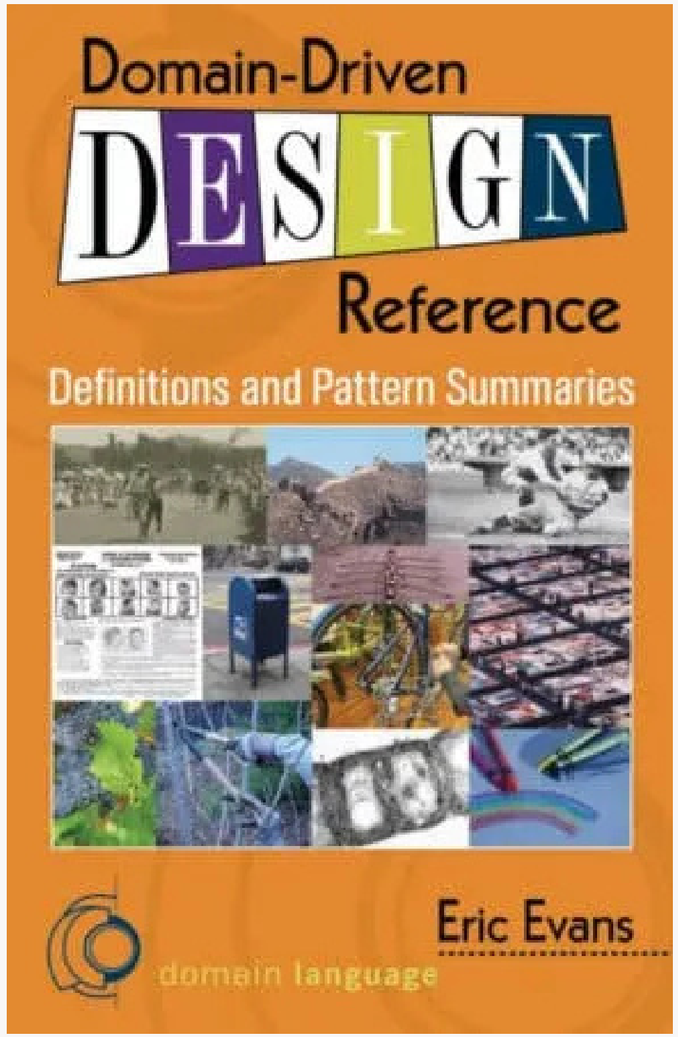
io.vavr.collection.List

```
1  import io.vavr.collection.List;
2
3  List<BoardMoveHint> moveHints = List.of(
4      new BoardMoveHint(1, 2, 10),
5      new BoardMoveHint(3, 4, 20)
6  );
7
8  List<BoardMoveHint> updatedMoveHints =
9      moveHints.append(new BoardMoveHint(5, 6, 30));
10
```

append() ne modifie pas *moveHints*,
elle **renvoie** une nouvelle liste *updatedMoveHints*

DDD (Domain-Driven Design)

Le DDD (Domain-Driven Design) est une approche de conception où on construit le logiciel autour du métier. L'objectif est que les classes Java représentent directement les concepts du domaine métier.



2 grandes familles de patterns

Patterns Strategies

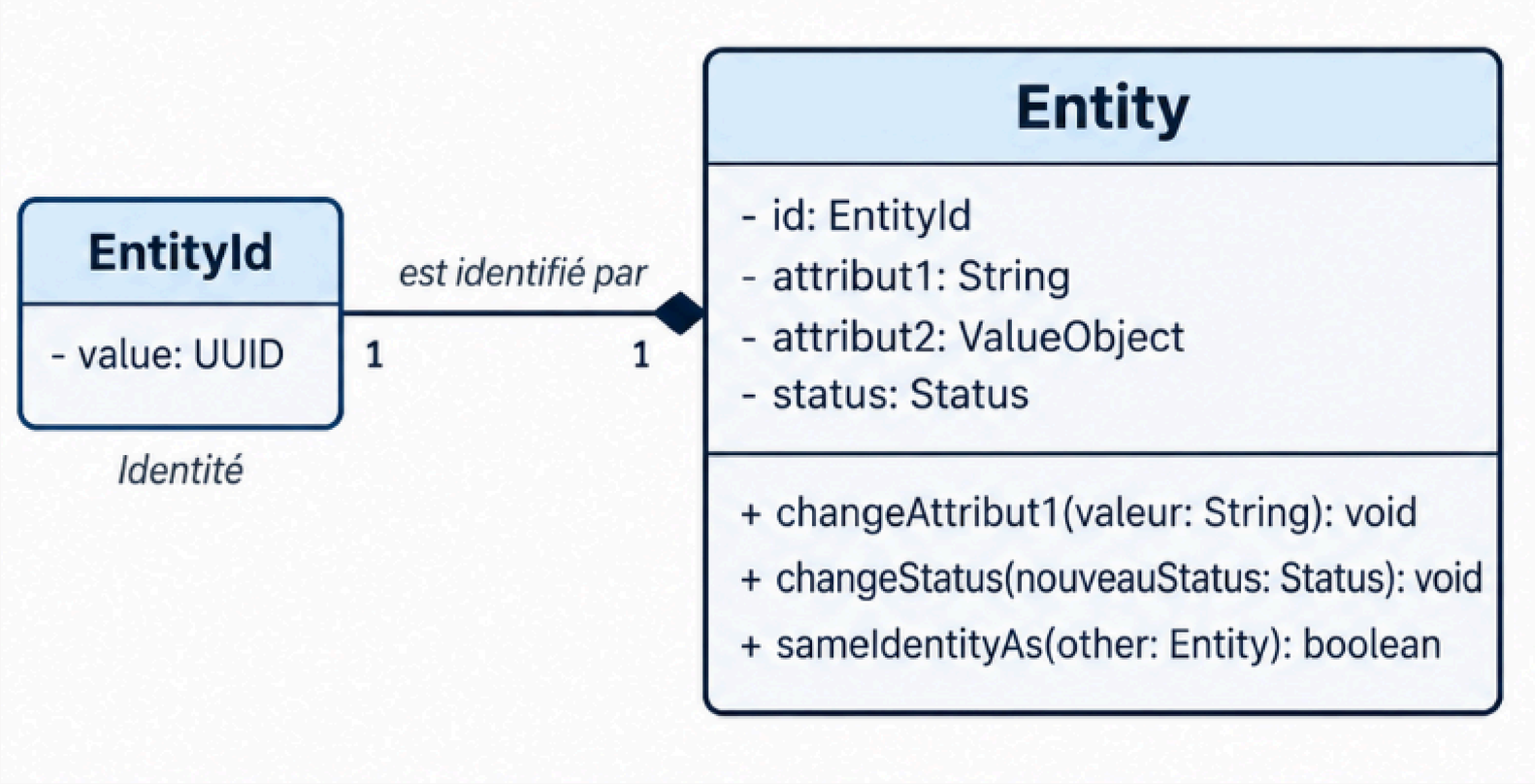
- Bounded Context
- Context Map
- Shared Kernel
- ...

Patterns Tactiques

- Entities
- Value Objects
- Aggregates
- ...

Qu’est-ce que le pattern Entitie ?

Le pattern Entity représente un objet métier qui possède une identité propre. Contrairement à un Value Object, une Entity peut évoluer dans le temps tout en restant le même objet.



Comment mettre en place le partern Entitie ?

Methode

1. **Définir une identité unique : par exemple un UUID id.**
2. **Conserver une identité stable : l'identifiant ne change pas pendant la vie de l'Entity.**
3. **Définir les attributs métier : par exemple name et email.**
4. **Permettre l'évolution de l'état : grâce à des méthodes métier comme changeName().**
5. **Redéfinir equals() et hashCode() : pour comparer les Entities par leur identité et non par tous leurs attributs.**
6. **Respecter les règles métier : éviter les modifications qui rendraient l'Entity invalide.**

Exemple pattern Entitie

```java
public class Client {

    private final int id;
    private String nom;
    private String email;

    public Client(int id, String nom, String email) {
        this.id = id;
        this.nom = nom;
        this.email = email;
    }

    public void changeName(String nouveauNom) {
        this.nom = nouveauNom;
    }

    public void changeEmail(String nouveauEmail) {
        this.email = nouveauEmail;
    }

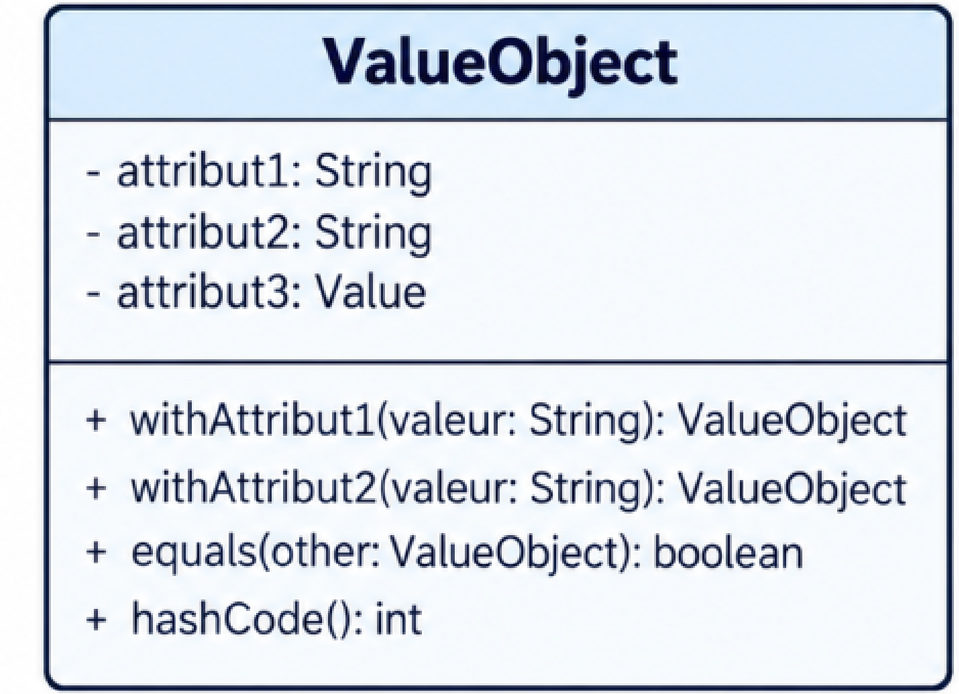
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Client other)) return false;

        return id == other.id;
    }

    @Override
    public int hashCode() {
        return Integer.hashCode(id);
    }
}
```

Qu’est-ce que le pattern Value Object?

Le pattern Value Object permet de représenter une valeur métier. Contrairement à une Entity, il ne possède pas d’identité propre : ce sont ses attributs qui le définissent.



Comment mettre en place le partern Value Object ?

Methode

1. **Pas d'identifiant unique** : l'objet est défini par ses attributs.
2. **Classe final** : pour empêcher l'héritage dans notre implémentation immuable.
3. **Attributs private final** : pour empêcher leur réaffectation.
4. **Aucun setter** : l'objet ne doit pas pouvoir être modifié après sa création.
5. **Validation dans le constructeur** : pour garantir un état valide dès la création.
6. **Redéfinition de equals() et hashCode()** : pour comparer les objets par leurs valeurs.

Exemple pattern Value Object

```java
public final class Billet {

    private final int montant;
    private final String devise;

    public Billet(int montant, String devise) {
        if (montant < 0 || devise == null || devise.isBlank()) {
            throw new IllegalArgumentException("Valeur invalide");
        }
        this.montant = montant;
        this.devise = devise;
    }
    public int getMontant() {
        return montant;
    }
    public String getDevise() {
        return devise;
    }
    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Billet autre)) return false;

        return montant == autre.montant
                && devise.equals(autre.devise);
    }
    @Override
    public int hashCode() {
        return 31 * Integer.hashCode(montant)
                + devise.hashCode();
    }
    @Override
    public String toString() {
        return montant + " " + devise;
    }
}
```

```java
public class Main {

    public static void main(String[] args) {

        Billet billet1 = new Billet(20, "EUR");
        Billet billet2 = new Billet(20, "EUR");
        System.out.println(billet1.equals(billet2));
        // true
    }
}
```

Exemple pattern Value Object avec Record

```java
public record Billet(double amount, String currency) {

    public Billet {
        if (amount < 0) {
            throw new IllegalArgumentException(
                "Le montant ne peut pas être négatif"
            );
        }

        if (currency == null || currency.isBlank()) {
            throw new IllegalArgumentException(
                "La devise est obligatoire"
            );
        }
    }
}
```

```java
public class Main {

    public static void main(String[] args) {

        Billet m1 = new Billet(20, "EUR");
        Billet m2 = new Billet(20, "EUR");

        System.out.println(m1.equals(m2));
        // true
    }
}
```

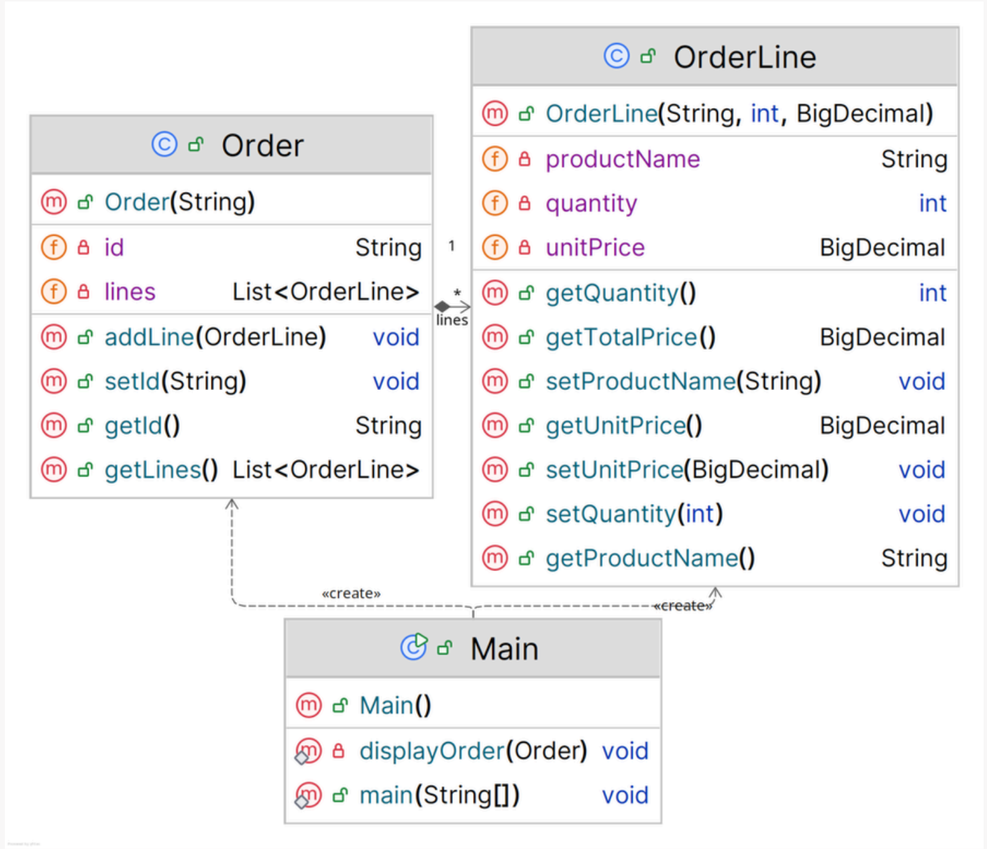
Entity vs Value Object

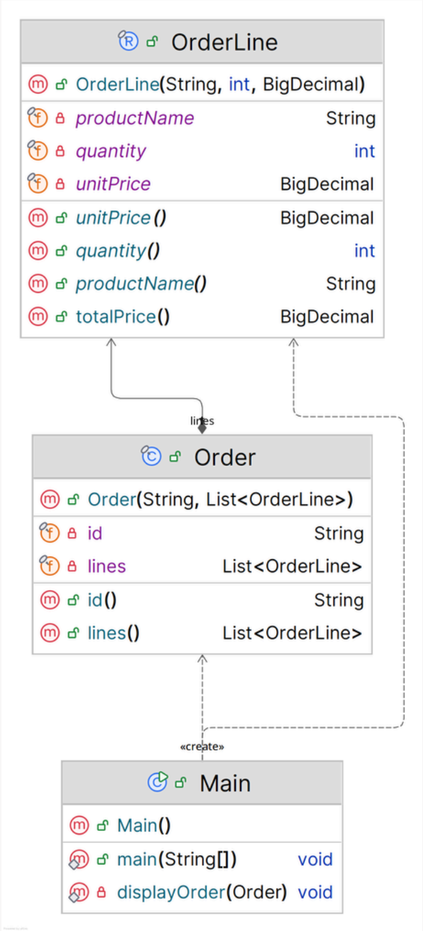
Critères	Entity	Value Object
Identité	Possède un identifiant unique	Pas d'identité propre
Égalité	Comparaison par identifiant	Comparaison par valeurs
Immuabilité	Généralement mutable	Immuable
Modification	Ses attributs peuvent évoluer	Création d'un nouvel objet
Cycle de vie	Suivi dans le temps	Pas de suivi individuel
En Java	class	final class ou record

LIVE CODING



LIVE CODING


Order
Order(String)
id String
lines List<OrderLine>
addLine(OrderLine) void
setId(String) void
getId() String
getLines() List<OrderLine>
1
*
lines
OrderLine
OrderLine(String, int, BigDecimal)
productName String
quantity int
unitPrice BigDecimal
getQuantity() int
getTotalPrice() BigDecimal
setProductName(String) void
getUnitPrice() BigDecimal
setUnitPrice(BigDecimal) void
setQuantity(int) void
getProductName() String
«create»
«create»
Main
Main()
displayOrder(Order) void
main(String[]) void


OrderLine
OrderLine(String, int, BigDecimal)
productName String
quantity int
unitPrice BigDecimal
unitPrice() BigDecimal
quantity() int
productName() String
totalPrice() BigDecimal
lines
Order
Order(String, List<OrderLine>)
id String
lines List<OrderLine>
id() String
lines() List<OrderLine>
«create»
Main
Main()
main(String[]) void
displayOrder(Order) void

QCM

SITOGRAPHIE/IAGRAPHIE

- https://docs.oracle.com/
- https://www.baeldung.com/
- https://stackoverflow.com/
- https://errorprone.info/

- claude.ai Relecture du diapo
- gemini.google.com Idée d’exemple